

USB 풋페달 클러치 가이드

📖 teleop 문서 진입점: [README.md](#) — 하려는 일부터 문서를 고른다.

USB HID 풋페달로 teleop 클러치를 조작한다. 밟는 동안만 리더 동작이 팔로워로 전달되는 **데드맨** 구성이 기본이다 (용어 설명: [부록 A](#)).

- 노드: [clutch_pedal_node.py](#) (ros2 run rdfp clutch_pedal)
- 클라이언트: [clutch_client.py](#) (ClutchClient)
- 클러치 노드: [teleop_retargate_node_guide.md](#)

1. 왜 hold(데드맨)인가

hold (밟는 동안만) toggle (밟을 때마다 전환)

조작자가 밟을 때면	정지	계속 물려 있음
조작자가 자리를 뜨면	정지	계속 물려 있음
"지금 물려 있나?" 기억 불필요		사람이 기억해야 함
장시간 피로	있음	없음

산업용 로봇 티치펜던트의 인에이블링 스위치가 hold 인 이유와 같다 — **놓았을 때 멈추는 쪽이 안전한 기본값**이다. teleop_retargate 의 클러치 극성이 "engaged = 따라감" 이므로 hold 로 두면 fail-safe 방향과 일치한다.

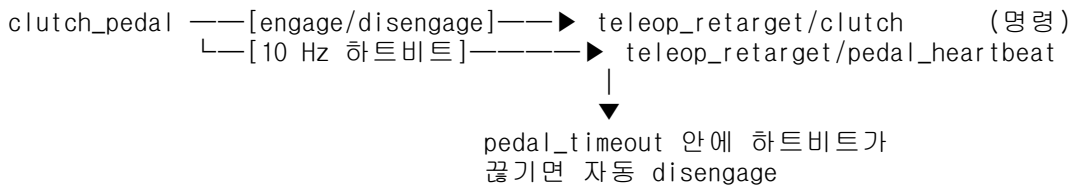
toggle 모드도 제공하지만 **데드맨이 아니다**. 피로가 문제되는 장시간 작업이나 안전 위험이 낮은 환경에서만 쓴다.

터미널 키보드(teleop_keyboard 방식)로는 hold 를 구현할 수 없다. raw/cbreak 모드는 키를 **땀 것을 알려주지 않기** 때문이다. USB HID 페달을 evdev 로 직접 읽어야 press/release 를 모두 받는다.

2. 데드맨을 완성하는 두 조각

페달 노드만으로는 부족하다. 노드가 죽거나 USB 가 빠지면 **disengage** 를 보낼 주체가 사라져 클러치가 물린 채로 남는다 — 데드맨이 막아야 할 상황을 정확히 못 막는다.

그래서 두 조각이 함께 있어야 한다.



조각	파라미터	없으면
페달 노드	mode:=hold	뎀을 감지 못 함
retarget 노드	pedal_timeout:=0.3	노드 사망·USB 분리 시 물린 채로 남음

pedal_timeout 기본값은 0(비활성)이다. 콘솔·GUI 로만 조작하는 기존 구성에 영향을 주지 않기 위해서다. 페달을 쓸 때는 반드시 켜야 한다.

3. 사전 준비

3-1. evdev 설치

```
sudo apt install python3-evdev
```

없으면 노드가 이렇게 알려주고 종료한다.

```
RuntimeError: python3-evdev is required for clutch_pedal_node.
Install it with: sudo apt install python3-evdev
```

3-2. 권한

/dev/input/event* 는 root:input 소유다. input 그룹에 속해야 읽을 수 있다.

```
id -nG | tr ' ' '\n' | grep -x input      # 없으면 아래 실행 후 재로그인
sudo usermod -aG input $USER
```

3-3. 장치 이름과 키코드 확인

페달을 꽂고 확인한다.

```
# 장치 이름 → § 4 의 pedal_device_name (ros2 run 은 device_name) 에 넣는다
grep -E "^N: Name" /proc/bus/input/devices
```

```
# 밟았을 때 나오는 키코드 → § 4 의 pedal_key_code (ros2 run 은 key_code) 에 넣는다
python3 -m evdev.evtest
```

장치 이름은 부분 문자열로 매칭(대소문자 무시)하므로 전체를 적을 필요는 없다.

N: Name="VEC VEC USB Footpedal" 이면 Footpedal 만으로 충분하다.

페달마다 보내는 키가 다르다 (KEY_A, BTN_LEFT, KEY_LEFTCTRL 등).

4. 실행

방법 A — 런치에 맡긴다 (권장)

`enable_clutch_pedal:=true` 를 주면 페달 노드까지 함께 뜬다. 터미널 하나면 된다.

```
ros2 launch rdfp teleop_mirror.launch.py W
enable_clutch_pedal:=true W
pedal_device_name:=pedal W
pedal_key_code:=KEY_A
```

⚠ `pedal / KEY_A` 는 예시일 뿐 특정 제품의 값이 아니다 — § 3-3 에서 확인한 값으로 바꾼다. `pedal` 은 `device_name` 의 기본값이라 이름에 `pedal` 이 들어가는 장치라면 인자 자체를 생략해도 된다.

`pedal_timeout` 을 안 줘도 된다 — 런치가 `hold` 모드일 때 0.3 을 자동으로 채워 넣는다 (§ 4-1). 다르게 잡고 싶을 때만 명시한다.

인자	YAML 키	기본값	
<code>enable_clutch_pedal</code>	<code>clutch_pedal.enabled</code>	<code>false</code>	페달 노드 기동 여부
<code>pedal_device_path</code>	<code>clutch_pedal.device_path</code>	<code>''</code>	장치 경로 (비우면 이름 탐색)
<code>pedal_device_name</code>	<code>clutch_pedal.device_name</code>	<code>pedal</code>	장치 이름 부분 문자열
<code>pedal_key_code</code>	<code>clutch_pedal.key_code</code>	<code>''</code>	키 이름 (비우면 아무 키나)
<code>pedal_mode</code>	<code>clutch_pedal.mode</code>	<code>hold</code>	<code>hold / toggle</code>
<code>pedal_grab</code>	<code>clutch_pedal.grab</code>	<code>false</code>	장치 독점
<code>pedal_timeout</code>	<code>clutch_pedal.timeout</code>	<code>0.0</code>	하트비트 감시 (자동 보정됨)

기본값이 `false` 인 이유는 `python3-evdev` 와 실제 장치가 없으면 노드가 기동에 실패하기 때문이다. 페달을 안 쓰는 구성에 부담을 주지 않는다.

4-0. 매번 인자를 치기 귀찮다면 — YAML 에 박아 둔다

`teleop_mirror.launch.py` 의 기본값은 `<rdfp share>/config/teleop_mirror.yaml` 에서 온다. 페달 설정을 고정해 두면 인자 없이 띄울 수 있다.

```
# config/teleop_mirror.yaml
clutch_pedal:
  enabled: true
  device_name: Footpedal    # § 3-3 에서 확인한 값
  key_code: KEY_A
  mode: hold
  grab: true
```

```
ros2 launch rdfp teleop_mirror.launch.py    # 인자 없이 페달까지 뜬다
```

소스는 src/rdfp/config/teleop_mirror.yaml 이고 빌드 시 share 로 복사되므로,
고친 뒤 colcon build 를 해야 반영된다. 빌드 없이 시험하려면 파일을 복사해
config_file:= 로 지정한다.

```
cp src/rdfp/config/teleop_mirror.yaml ~/my_pedal.yaml # 편집 후
ros2 launch rdfp teleop_mirror.launch.py config_file:=$HOME/my_pedal.yaml
```

CLI 인자가 YAML 보다 우선하므로, YAML 로 고정해 두고 그때그때 pedal_mode:=toggle
같은 식으로 덮어쓰는 것도 된다.

방법 B — 따로 띄운다

디버깅이나 파라미터를 바꿔가며 시험할 때 편하다. 이때는 pedal_timeout 을
직접 켜야 한다.

```
# 터미널 A
ros2 launch rdfp teleop_mirror.launch.py pedal_timeout:=0.3

# 터미널 B - device_name / key_code 는 § 3-3 에서 확인한 값
ros2 run rdfp clutch_pedal --ros-args W
  -p device_name:=pedal -p key_code:=KEY_A -p mode:=hold
```

런치 없이 teleop_retarget 만 직접 띄워도 된다.

```
ros2 run rdfp teleop_retarget --ros-args W
  -p leader_pose_topic:=/leader/ee_pose W
  -p follower_pose_topic:=/ee_pose W
  -p pedal_timeout:=0.3
```

4-1. 런치의 자동 보정

데드맨은 **페달 노드**(땜 감지)와 pedal_timeout(하트비트 감시) 두 조각이
함께 있어야 성립한다. 한쪽만 켜 구성은 조용히 잘못 동작하므로 런치가 정합을
맞춘다.

입력	결과 pedal_timeout	이유
enable_clutch_pedal:=true, hold, timeout 미지정	0.3	데드맨 아닌 채로 뜨는 것을 방지
enable_clutch_pedal:=true, hold, timeout:=0.5	0.5	명시값 존중
pedal_mode:=toggle, timeout:=0.3	0.0	toggle 은 하트비트를 안 보내므로 켜면 engage 즉시 풀림
enable_clutch_pedal:=false, timeout:=0.3	0.3	페달 노드를 따로 띄우는 구성 (방법 B)

설정이 반영됐는지 확인:

```
ros2 param get /teleop_retargget pedal_timeout      # Double value is: 0.3
ros2 topic info /teleop_retargget/pedal_heartbeat  # Subscription count: 1
```

pedal_timeout 이 0 이면 **하트비트 토픽을 구독하지도 않는다** (Subscription count: 0). 이것으로 켜졌는지 바로 판별할 수 있다.

동작 확인:

```
ros2 topic echo /teleop_retargget/clutch_state      # 밟고 떼면 engaged 가 바뀐다
```

5. 파라미터

이름	기본값	설명
device_path	''	장치 경로. 비우면 device_name 으로 탐색
device_name	'pedal'	장치 이름 부분 문자열 (대소문자 무시)
key_code	''	대상 키 이름. 비우면 아무 키나 페달로 취급
mode	'hold'	hold (데드맨) / toggle
heartbeat_rate	10.0	hold 모드 하트비트 주기(Hz). pedal_timeout 보다 충분히 빨라야 한다
clutch_node_name	'teleop_retargget'	클러치 노드 이름 (서비스·토픽 prefix)
grab	False	True 면 장치를 독점(EVIOCGRAB)해 키 입력이 데스크톱·터미널로 새지 않는다
reconnect_interval	2.0	장치 재탐색 주기(초). 0 이하면 재시도 안 함

device_path VS device_name

/dev/input/eventN 번호는 **재부팅·재연결 시 바뀐다**. 고정하려면 둘 중 하나를 쓴다.

```
-p device_path:=/dev/input/by-id/usb-XXXX-event-kbd    # 심볼릭 링크 (권장)
-p device_name:=pedal                                   # 이름 매칭
```

grab 를 켜는 경우

페달이 일반 키보드로 인식되면 밟을 때마다 **터미널이나 데스크톱에 문자가 입력된다**. grab:=true 로 장치를 독점하면 이 노드만 이벤트를 받는다.

key_code 를 비우면

아무 키나 페달로 취급한다. 페달이 단일 키만 보내는 것이 확실할 때 편하지만, 일반 키보드를 잘못 잡으면 모든 타이핑이 클러치를 조작하게 된다. 장치를 확실히 지정한 경우에만 비워 둔다.

6. 안전 동작

상황	동작
페달 땜	즉시 disengage (hold 모드)
USB 분리	장치 읽기 실패 감지 → 즉시 disengage + 재탐색
노드 정상 종료	shutdown() 이 disengage 호출
노드 크래시 / SIGKILL	하트비트 끊김 → pedal_timeout 후 자동 해제
리더 스트림 끊김	watchdog_timeout 후 자동 해제 (기존 기능)

크래시 경로는 pedal_timeout 만이 막는다. 종료 혹은 정상 종료에서만 동작하기 때문이다.

키 자동반복(value=2)은 무시한다 — 새 누름으로 오인하면 안 되기 때문이다.

7. ClutchClient — 코드에서 클러치 다루기

GUI 나 다른 노드에서 클러치를 쓸 때 쓴다. 서비스 2개와 상태 구독을 캡슐화한다.

```
from rdfp.teleop.clutch_client import ClutchClient

client = ClutchClient.create(node)                # node_name 기본 'teleop_retarget'
client.on_change(lambda engaged, reason: ...)      # 등록 즉시 현재 상태로 1회 호출

client.engage_async()                             # 비동기 (spin 중인 노드용)
client.disengage_async()
client.toggle_async()                             # 캐시된 상태의 반대로

client.engaged                                    # 마지막 수신 상태 (None = 미수신)
client.reason                                     # 마지막 해제 사유
client.get_state(timeout_sec=3.0)                 # 동기 조회
```

직접 구현할 때 틀리기 쉬운 두 가지를 감춘다.

1. clutch_state 는 TRANSIENT_LOCAL 이다. 기본 QoS 로 구독하면 래치된 값을 못 받아 **조용히 아무것도 안 온다** (GUI 램프가 계속 회색).
2. 동기 API 는 spin_until_future_complete 를 쓴다. 이미 spin 중인 노드 (Tk mainloop, 키 입력 루프)에서 부르면 데드락이다 → *_async 를 쓴다.

on_change 는 등록 시점에 이미 상태를 받아 두었으면 **즉시 한 번 호출**한다.
GUI 가 늦게 붙어도 현재 상태를 바로 그릴 수 있다.

8. 문제 해결

페달을 밟아도 아무 반응이 없다

```
# 노드가 장치를 잡았는지
ros2 node list | grep clutch_pedal
# 로그에 'pedal connected: /dev/input/eventN (...)' 가 있어야 한다
```

없으면 device_name 매칭 실패다. `grep -E "^N: Name" /proc/bus/input/devices` 로 실제 이름을 확인한다.

장치는 잡았는데 반응이 없으면 key_code 가 틀린 것이다. key_code 를 비워 (-p key_code='') 아무 키나 받게 해 보고, 로그에 pedal pressed 가 뜨는지 확인한다.

engage 되자마자 바로 풀린다

```
reason: pedal heartbeat never received
```

pedal_timeout 은 컷는데 하트비트가 안 오는 상황이다. 원인은 둘이다.

- 페달 노드가 toggle 모드다 → toggle 은 하트비트를 보내지 않는다
- clutch_node_name 이 틀려 하트비트가 다른 토픽으로 나간다

```
ros2 topic hz /teleop_retarget/pedal_heartbeat    # 밟고 있는 동안 10 Hz
```

밟는 도중에 자주 풀린다

```
reason: pedal heartbeat stale for 0.31s
```

heartbeat_rate 가 pedal_timeout 에 비해 느리다. 주기의 3배 이상을 타임아웃으로 잡는 것이 안전하다 (10 Hz → 0.3초).

밟을 때마다 터미널에 문자가 찍힌다

페달이 일반 키보드로 인식된 것이다.

```
-p grab:=true
```

권한 오류

```
cannot open /dev/input/eventN: [Errno 13] Permission denied
```

input 그룹에 없다. § 3-2 참고. `usermod` 후 재로그인해야 반영된다.

부록 A. 데드맨 스위치란

용어만 확인하려면 여기부터 읽어도 된다. 본문은 이 개념을 전제로 쓰여 있다.

데드맨 스위치(dead-man switch) 는 조작자가 계속 붙잡고 있어야만 동작하고, 손을 놓거나 조작자에게 이상이 생기면 **자동으로 정지**하는 안전 장치다. 이름은

"조작자가 쓰러져도(dead man) 기계가 안전하게 멈춘다" 는 뜻에서 왔다.

A-1. 유래

기관차 운전실의 페달·레버가 원형이다. 기관사가 밟고 있어야 열차가 달리고, 기절해 발이 떨어지면 비상 제동이 걸린다. 오늘날은 잔디깎기 손잡이(놓으면 날 정지), 체인톱, 산업용 로봇 티치펜던트의 인에이블링 스위치 등에 쓰인다.

A-2. 핵심 원리 — 정지가 기본값

일반 스위치	데드맨
ON 유지	아무것도 안 해도 유지 계속 힘을 줘야 유지
조작자 이상 ON 상태로 방치	자동 OFF
신호가 끊기면 마지막 상태 유지	정지

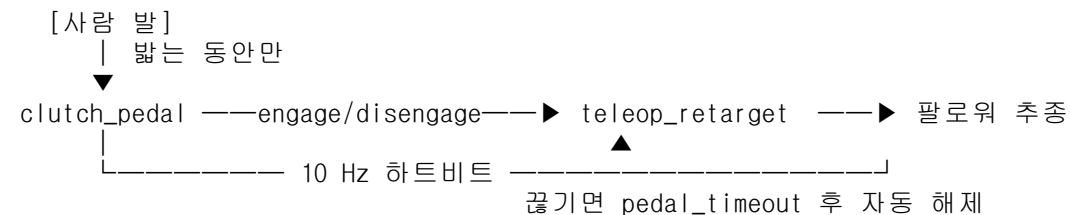
핵심은 "움직임"이 예외 상태이고 "정지"가 기본 상태라는 점이다. 아무 일도 일어나지 않으면 — 사람이 손을 놓든, 선이 끊기든, 프로세스가 죽든 — 정지 쪽으로 떨어진다. 이것이 fail-safe(고장 시 안전) 설계다.

그래서 데드맨이 성립하려면 세 가지가 모두 필요하다.

조건	이 프로젝트에서
1. 뽕을 감지할 수 있어야 한다	evdev release 이벤트 (mode:=hold)
2. 한 번 켜면 래치되지 않아야 한다	hold 모드 — toggle 은 래치되므로 실격
3. 신호 소실도 정지로 해석해야 한다	하트비트 + pedal_timeout

3번이 빠지기 쉽다. 페달을 밟은 채 노드가 죽으면 1·2번이 아무리 정확해도 해제 명령을 보낼 주체가 사라지기 때문이다. 본문 § 2 가 "두 조각"이라고 부르는 것이 이것이다.

A-3. 이 프로젝트의 데드맨



끊기는 지점	막는 장치
발을 뽕	mode:=hold (release 이벤트)
USB 분리	읽기 실패 감지 → 즉시 disengage
노드 정상 종료	shutdown() 후

끊기는 지점

막는 장치

노드 크래시 / SIGKILL 하트비트 소실 → `pedal_timeout`

리더 스트림 끊김 `watchdog_timeout`

A-4. 데드맨이 아닌 것

- `mode:=toggle` — 밟을 때마다 on/off 가 래치되므로 A-2 의 2번을 위반한다. 발을 떼도 계속 물려 있다. 편의 기능이지 안전 장치가 아니다.
- **비상정지(E-stop) 버튼** — 목적이 반대다. E-stop 은 "이상을 발견했을 때 사람이 누르는" 것이고, 데드맨은 "사람이 **아무것도 못 하게 됐을 때** 알아서 멈추는" 것이다. 둘은 대체 관계가 아니라 보완 관계다.
- `pedal_timeout:=0` **인 hold 구성** — 겉보기엔 데드맨이지만 크래시 경로가 열려 있다. 런치가 이 조합을 자동으로 보정하는 이유다 (§ 4-1).

A-5. 참고 — 산업용 3-포지션 스위치

로봇 티치펜던트의 인에이블링 스위치(IEC 60204-1 / ISO 10218 계열)는 데드맨을 한 단계 확장한 형태다.

위치	상태
놓음	OFF
살짝 누름(중간)	ON
꽂 누름	OFF

놀라서 움켜쥐는 반응(startle reflex)까지 정지로 처리하기 위해서다. 풋페달은 2-포지션이라 이 보호는 없다 — 위험 작업에서는 물리적 E-stop 을 별도로 둔다.

관련 문서

- [teleop_retargate_node_guide.md](#) — 클러치 상태 기계, `pedal_timeout` 을 포함한 파라미터 전체
- [omy_leader_teleop_guide.md](#) — 전체 체인 기동 절차
- [leader_follower_mirroring_design.md](#) — 안전장치 설계 배경